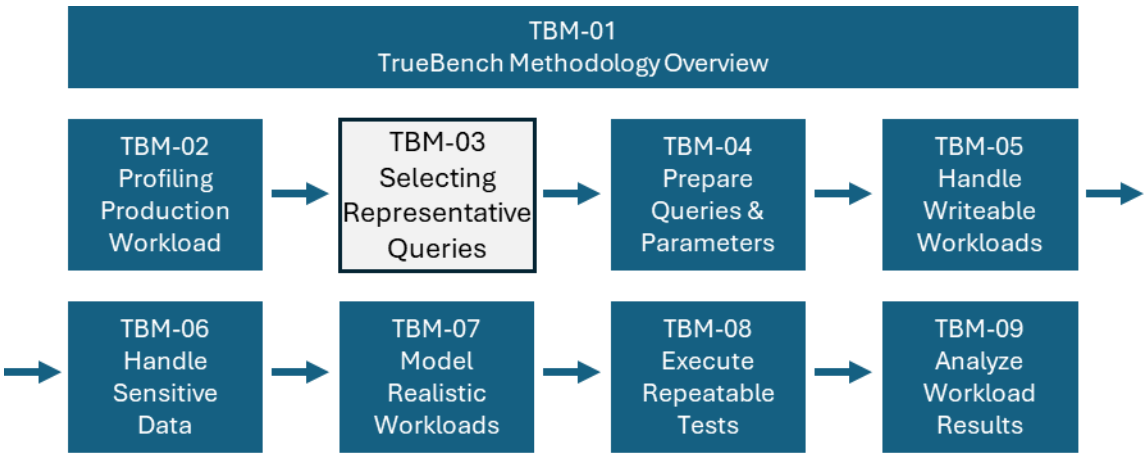
Selecting Representative Queries

Reducing Millions of Queries to a Representative Test Workload.

Author: Douglas H Ebel

Data Warehouse Architect
Business Value and Performance Assessment

Creator of TdBench and TrueBench



Introduction

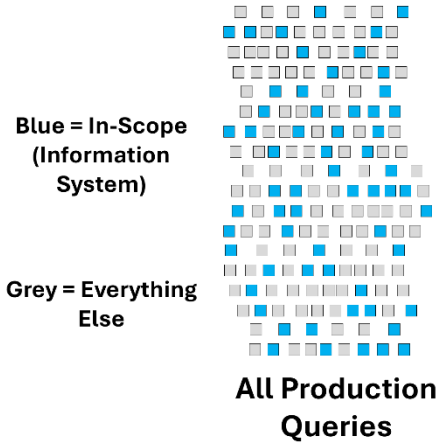
In TBM-02, you profiled the Information System workload. In a month, there may have been tens of thousands of queries executed. You now need to select a subset that is:

- representative of the information system's usage of data and system components
- small enough to fit into a test window that won't impact production
- of a practical size for you to prepare without making it a new full-time job

People who define these types of tests tend to fall into one of three categories:

- they know something about the information system
- they think they know something about the information system
- or, what was often the case in my experience, they know nothing about the information system

We spent time in TBM-02 building a workload profile that we now need to match. That profile tells us what "normal" looks like and provides strong hints about what needs to be included in the test workload.



To accomplish the selection of the test workload, we are going to:

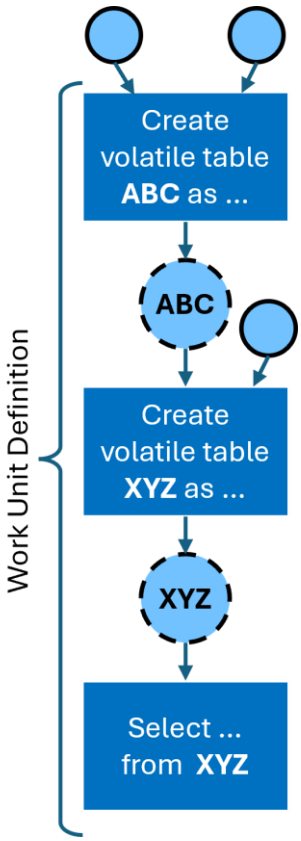
1. Identify the databases that are in scope
2. Select candidate hours from the profile as the source of queries
3. Match the profile of selected queries to production and expand the selection if needed
4. Remove queries that would impact production or prevent repeatable execution in the future
5. Determine whether queries can be executed independently or must be executed as part of a script
6. Introduce considerations for maintaining production data during testing (covered at a high level in this document)

Steps 4 and 5 require a bit of explanation before we begin.

For tests to be repeatable, queries must reference tables that will be present when the test is rerun. Queries that depend on worktables or user-specific tables may fail in the future if those objects are unavailable.

Updates to production tables introduce additional concerns. They not only pose a risk to production integrity during testing, but also require carefully constructed transaction data to ensure that updates remain valid when the test is rerun. This adds significant complexity and is typically out of scope for initial validation efforts.

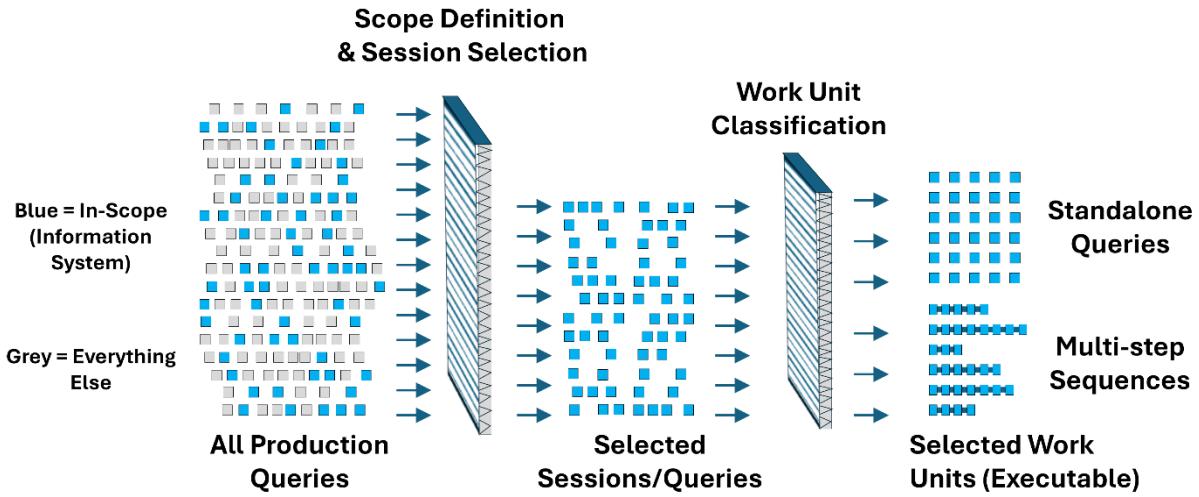
At the same time, some scripts generated by tools use intermediate tables in their logic. These create/insert/update/delete operations can be included in the test workload, but they must be handled carefully. If these scripts are executed concurrently, they should use volatile tables rather than shared worktables to avoid conflicts between sessions.



As a result, queries will be selected by session, and during this selection process, we will identify:

- queries that can be executed independently as standalone work units
- queries that must be executed consecutively as part of a multi-step sequence
- queries that must be excluded due to references outside the defined database scope or because they impact production tables

The steps described in this document are implemented as a pipeline that uses a small number of tables to capture intermediate results from the selection process. These tables allow the workload to be reviewed, refined, and reproduced as it is transformed from production query logs into executable work units. The data model for these tables is provided in Appendix A.



The goal is not simply to reduce the number of queries. The goal is to define a workload that is representative, repeatable, and practical to prepare and execute. Considerations for simulating production data maintenance during test execution are significant and will be introduced at a high level here. A more complete treatment of this topic will be covered separately.

Defining the Scope of the Information System

Before selecting queries, you need to define the boundary of the information system you are validating.

In TBM-02, the workload profile was built using queries executed by a defined set of users. Those same queries can be used to identify the databases referenced during normal system activity. By analyzing the query log and associated object references, you can derive a list of databases that represent the information system.

From that list, you define a simple control table that identifies which databases are:

- **in scope (Y)** for testing
- **out of scope (N)** and excluded
- **work or staging databases (W)** that require review

This step is important because production platforms often contain:

- personal databases
- work or staging databases
- data sets used for one-time analysis
- objects unrelated to the information system being validated

Including these objects in the test workload can introduce queries that:

- cannot be reproduced in the future
- depend on data or structures that no longer exist
- distort the behavior of the workload

For this reason:

- queries referencing **in-scope databases (Y)** are eligible for automatic inclusion
- queries referencing **out-of-scope databases (N)** should be excluded
- queries referencing **work databases (W)** should be retained for review, as they may be candidates for conversion to volatile tables or inclusion as part of multi-step work units

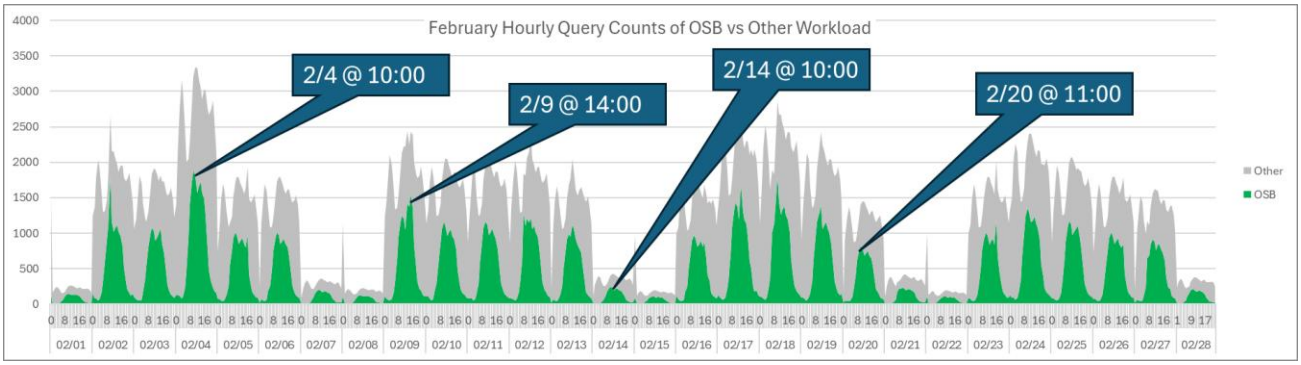
The result of this step is a control table of databases that is used throughout the remainder of the selection and preparation process to drive filtering and review decisions.

The example SQL used to derive this list is provided in Appendix B.

Selecting Candidate Sessions from the Workload Profile

In TBM-02, you developed a workload activity profile over time. That profile offers a practical way to choose a manageable subset of queries while maintaining the characteristics of the production workload.

Instead of choosing individual queries, we start by selecting activity hours from the profile. Each chosen hour offers a set of sessions and queries that show how the system was used during that time. Sample SQL used for the selection and profiling process is included in Appendix C.



The graph above shows hourly query counts for the OSB workload compared with other platform activities. Based on this profile, four representative hours were chosen:

- February 4 at 10:00
- February 9 at 14:00
- February 14 at 10:00
- February 20 at 11:00

These hours were selected to reflect variation in workload patterns throughout the month, including differences in volume and usage behaviors.

From these four hours, the resulting selection included 3,560 OSB queries out of approximately 292,000 OSB queries executed during the month, and over 900,000 total queries on the platform. This represents roughly 1.2% of the OSB workload.

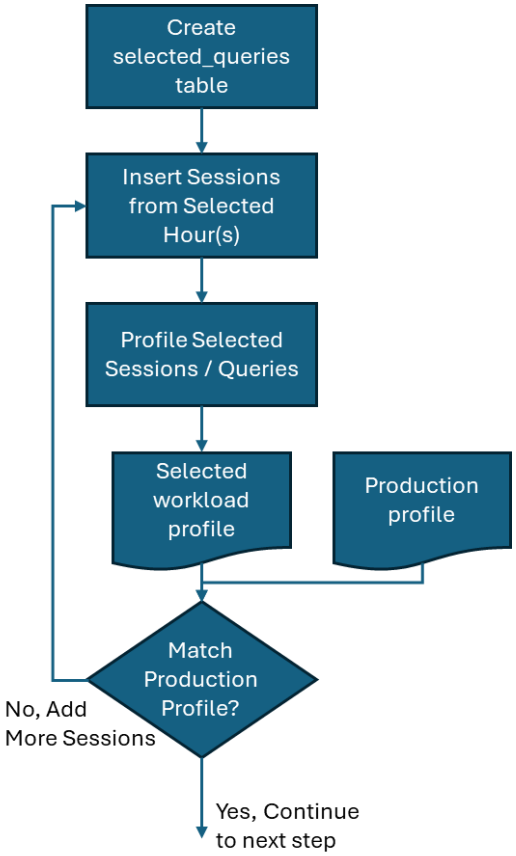
This initial reduction significantly limits the volume of queries that must be analyzed and prepared, while still preserving a representative sample of system activity.

At this stage, the goal is not to produce the final workload. The selected hours provide a candidate set of sessions and queries, which will be refined in subsequent steps to ensure

the workload remains representative of production behavior. Profiling queries used for validation are included in Appendix C.

Validating the Selected Workload Against the Profile

The selected hours provide a manageable starting point, but they do not guarantee that the resulting workload is representative of production behavior. Validation against the workload profile created in TBM-02 is required.



Using the same profiling approach, the queries in the selected_queries table are analyzed and compared to the production workload. The objective is to confirm that the selected workload reflects the characteristics observed in production, including:

- distribution of query complexity
- frequency of table access
- relative mix of query types
- overall volume patterns

In most cases, the initial selection will not perfectly match the production profile. Certain query types or table access patterns may be underrepresented, particularly if they occur outside the selected hours or at lower frequencies.

Appendix C shows queries for selecting hours to include in the selected_queries table; however, you may want to add additional logic to target specific query types. It also shows modified profiling queries, allowing you to compare the selected sessions/queries with the production profile.

Screening Queries for Reuse and Repeatability

At this stage, the selected_queries table contains a candidate set of sessions and queries that match the production workload profile. The next step is to refine that selection to ensure the workload can be prepared and executed in a repeatable and practical way.

This is not a single pass. It is an iterative process of reviewing, filtering, and occasionally re-profiling the selected workload.

The screening process focuses on four areas:

Out-of-Scope Database References

Queries that reference databases outside the defined information system should be excluded or flagged for review.

These references often point to:

- personal databases
- work or staging areas
- temporary or one-time analysis

Including these queries can introduce dependencies that cannot be reproduced when the test is rerun.

Tool-Generated Noise

Some tools generate queries that do not represent meaningful workload activity. These may include:

- metadata lookups
- repeated small queries used for navigation or validation
- system-generated statements that do not contribute to user-facing results

These queries can inflate query counts without contributing to workload behavior and should be removed from the selection. A simple update query against the selected_queries table could set all of these queries to be excluded if they are not material to the testing.

Maintenance of Production Tables

Queries that modify production tables require careful consideration.

As discussed in the Introduction, updates, inserts, and deletes to production tables pose risks to production data integrity and require additional effort to create valid, repeatable test conditions. In most cases, these sessions should be excluded from the workload.

Where intermediate tables are used within a session, those operations may be retained as part of a multi-step sequence, provided they do not impact shared or persistent data structures.

Reduction of Session Volume

Even after choosing representative hours and matching the workload profile, the number of sessions might still be more than is practical to prepare.

In these cases, an additional reduction step may be required. One approach is to order sessions by resource usage, such as CPU, and select a subset at regular intervals. For example, selecting approximately every *n*th session from an ordered list can reduce volume while preserving a range of workload characteristics.

The objective is not random reduction, but controlled reduction that maintains diversity in query behavior.

Review and Iteration

The screening process is typically performed using a SQL GUI, with queries ordered by session and query sequence. The include_flag in the selected_queries table can be used to include or exclude queries as decisions are made.

Appendix D provides example SQL to:

- update the include_flag based on common filtering conditions
- select subsets of sessions using techniques such as modulo-based sampling
- support iterative refinement of the selected workload

As queries are excluded or sessions are reduced, it is important to periodically re-profile the remaining workload to ensure that it continues to reflect the characteristics of the production profile.

Summary

The result of this step is a refined set of sessions and queries that:

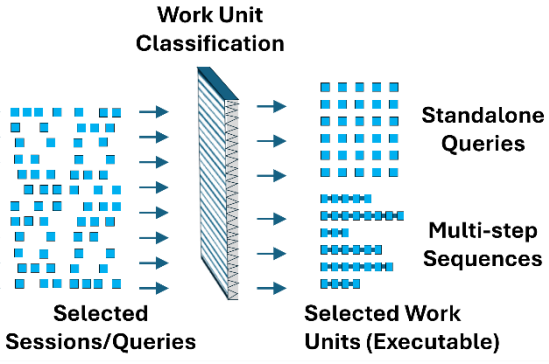
- reference only in-scope databases
- represent meaningful workload activity

- can be executed without impacting production data
- are of a practical size to prepare and manage

This refined set forms the input to the next step, where sessions are analyzed and transformed into executable work units.

Defining Executable Work Units

At this point, the selected_queries table contains a refined set of sessions and queries for constructing the test workload. The next step is to define the **executable work units** that will be prepared and run as part of the test.



Work units represent the smallest set of queries that must be executed together to preserve correct behavior.

The classification is performed at the session level, using the following rules.

Sessions with Volatile Tables

If a session creates a volatile table and subsequent queries reference that table, the entire session should be treated as a single work unit.

These queries are dependent on intermediate results created within the session and must be executed in sequence to preserve correct behavior.

Sessions Using Worktables

If a session creates, inserts into, updates, or deletes from tables in a work database, the queries may still be usable, but they require modification.

In these cases, the worktables should be converted to volatile tables so that multiple test sessions can run concurrently without conflict. Once converted, the queries in the session can be preserved as a multi-step sequence within a single work unit.

Independent Queries

If a session does not create or modify intermediate tables and all referenced objects are in scope, the queries can be treated as independent.

In these cases, individual queries may be separated and defined as standalone work units.

Special Case: Session-Scoped Parameter Tables

Some sessions create tables at the beginning of the session that act as parameter tables for subsequent queries. These tables may be created in the user's default database and referenced without qualification.

This pattern can be difficult to detect automatically, particularly when unqualified table names are used and the default database changes during the session. While systems such as Teradata record the default database for each query, identifying and converting these patterns reliably requires additional analysis.

As a result, this case is typically handled through manual review rather than automated classification.

Creating the Work Units

Based on this classification, the selected_queries table is transformed into the selected_work table.

- Each **standalone query** becomes a work unit with a single step
- Each **multi-step session** becomes a work unit with an ordered sequence of queries

The work_id identifies the work unit, and work_seq preserves execution order within the unit.

Summary

The objective of this step is to define executable units that preserve the original workload's logical behavior while allowing the test to be run repeatedly and, where appropriate, concurrently.

The result is a set of work units that:

- maintain required dependencies within sessions
- isolate independent queries for flexible execution
- avoid conflicts between concurrent test sessions

These work units form the input to the preparation step, where queries are parameterized and supporting data is generated.

Considerations for Simulating Data Maintenance

Most information systems are not read-only. They include ongoing inserts, updates, and deletes that affect the system's behavior under load.

Including data maintenance in a workload test is possible, but it **significantly increases the effort required to construct and manage the test**.

In some cases, this effort can be avoided. If data maintenance occurs during off-hours, or is handled by separate systems that do not interact with the workload being tested, it may be reasonable to exclude it from the initial validation effort.

This section provides a high-level overview of the considerations. A complete treatment of this topic warrants a separate methodology document.

Simplified Approaches

In one engagement, a large finance company simplified the problem by identifying two primary types of data maintenance:

- batch inserts from another system
- single-row updates driven by online activity

They created a copy of a key table and simulated maintenance activity by:

- running periodic batches of inserts during the workload test
- pacing single-row updates throughout the test

Both were controlled using workload pacing (implemented at the time using TdBench). This approach allowed them to introduce maintenance activity without replicating the full complexity of production processes.

Use of "Dummy" Data

Some systems include designated "dummy" customers, products, or transactions to support testing.

While this can simplify test construction, it has limitations:

- activity is confined to a subset of the data
- it may not reflect real access patterns
- it is less likely to create contention with read-only queries

As a result, this approach may not fully represent production behavior.

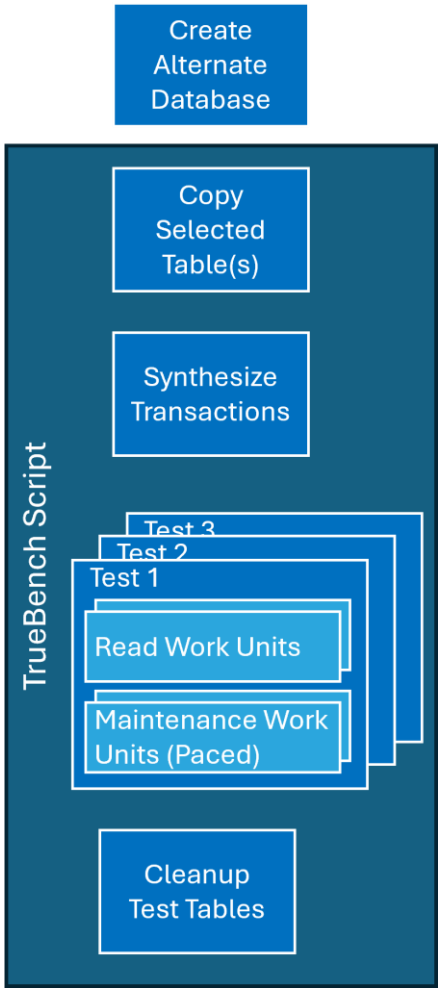
Using Production Maintenance Patterns

To simulate data maintenance more realistically, the following approach can be used:

1. **Isolate write activity**
   Create alternate databases to hold copies of tables that will be maintained.
   TrueBench sessions should have read-only access to production tables and write access only to the alternate databases.

2. **Prepare table copies**
   Copy selected tables prior to test execution.
   This can be automated using OS and SQL commands in the TrueBench script.
   One limitation is that copied tables may not reflect the physical characteristics of production data, such as fragmentation from ongoing updates.

3. **Synthesize transactions**
   Generate insert and update transactions based on existing data.
   This includes creating new keys and modifying values to maintain data integrity.



These transactions can be stored in parameter files and applied during test execution.

4. **Determine activity rates**
   Analyze production workload to estimate insert and update frequencies.
   These rates are used to define pacing for maintenance activity.

5. **Simulate contention**
   To reflect the interaction between maintenance and read activity, read-only work units may need to reference views that point to the alternate tables being updated.

Summary

Simulating data maintenance can improve the realism of a workload test, but it introduces additional complexity in data preparation, execution, and validation.

It is worth repeating:

- Simulating data maintenance is a major increase in the effort required to construct tests
- **Use extreme care in granting access to TrueBench** to production data to prevent accidental corruption.

For many validation efforts, it is appropriate to begin with read-only workloads and introduce data maintenance in later phases as needed.

Key Takeaways

- The goal is not to reduce query count, but to define a workload that remains representative of how the information system is used.
- Begin by defining scope. Only queries referencing in-scope databases should be included automatically.
- Selecting activity by hour provides a practical way to capture realistic usage while reducing volume.
- Validate the selected workload against the production profile. This is an iterative process of profiling, comparing, and refining.
- Screen out queries that cannot be reproduced, do not represent meaningful activity, or would impact production data.

- Classify sessions into executable work units: standalone queries or multi-step sequences.
- The result is a workload that can be prepared, executed, and repeated to validate system behavior.
- Simulating data maintenance improves realism but significantly increases effort; start with read-only workloads when practical

Appendix A – Selection Data Model

database_scope Table

This is the control table defining the information system boundary.

Recommended Definition

```
CREATE TABLE scope_databases (
database_name VARCHAR(128) NOT NULL,
in_scope_flag CHAR(1) NOT NULL,
scope_note VARCHAR(256),

CONSTRAINT pk_scope_databases
PRIMARY KEY (database_name),

CONSTRAINT ck_scope_in_scope_flag
CHECK (in_scope_flag IN ('Y', 'N', 'W'))
);
```

Comments

- in_scope_flag = Y or N. If a database is used for work tables, You could use the value of "W" to flag sessions using permanent work tables that need to be converted to volatile tables.
- scope_note is optional but useful for things like:
    - core subject area
    - personal database
    - work database
    - external

---

selected_queries Table

This is the TBM-03 input after selecting the representative hours/sessions and extracting the SQL.

Recommended columns

```
CREATE TABLE selected_queries (
session_id VARCHAR(64) NOT NULL,
logon_ts TIMESTAMP NOT NULL,
query_id VARCHAR(64) NOT NULL,
query_seq INTEGER NOT NULL,
```

```
include_flag CHAR(1) DEFAULT 'Y' NOT NULL,
sql_text CLOB NOT NULL,
review_status VARCHAR(32) DEFAULT 'AUTO_INCLUDED',
review_note VARCHAR(512),

CONSTRAINT pk_selected_queries
PRIMARY KEY (session_id, logon_ts, query_id),

CONSTRAINT ck_selected_queries_include_flag
CHECK (include_flag IN ('Y', 'N'))
);
```

Why these columns

- session_id, logon_ts together protect against reused session IDs
- query_id identifies the query
- query_seq preserves order within the session
- sql_text is needed for review and later processing
- include_flag allows default include/exclude
- review_status supports workflow
- review_note supports manual notes

Suggested meanings

- include_flag: Y / N
- review_status:
  - AUTO_INCLUDED
  - REVIEW_REQUIRED
  - MANUAL_INCLUDED
  - MANUAL_EXCLUDED

---

selected_work Table

This is the real TBM-03 output and TBM-04 input.

Recommended columns

```
CREATE TABLE selected_work (
work_id INTEGER NOT NULL,
session_id VARCHAR(64) NOT NULL,
logon_ts TIMESTAMP NOT NULL,
query_id VARCHAR(64) NOT NULL,
query_seq INTEGER NOT NULL,
```

```
work_seq INTEGER NOT NULL,
include_flag CHAR(1) DEFAULT 'Y' NOT NULL,
sql_text CLOB NOT NULL,

work_type VARCHAR(16) NOT NULL,

review_status VARCHAR(32) DEFAULT 'AUTO_INCLUDED',
review_note VARCHAR(512),

CONSTRAINT pk_selected_work
PRIMARY KEY (work_id, work_seq),

CONSTRAINT ck_selected_work_include_flag
CHECK (include_flag IN ('Y', 'N')),

CONSTRAINT ck_selected_work_type
CHECK (work_type IN ('STANDALONE', 'MULTISTEP'))
);
```

Why these columns

- work_id identifies the executable unit
- work_seq preserves order inside the work unit
- work_type tells whether this is:
    - STANDALONE
    - MULTISTEP
- keeping sql_text here avoids repeated joins during review
- same include/review columns allow manual override

Important behavior

For a standalone query:

- one row
- work_seq = 1

For a multi-step sequence:

- multiple rows
- same work_id
- work_seq = 1,2,3...

Appendix B — Identifying In-Scope Databases

The first step in workload selection is to identify the databases referenced by the information system users included in the workload profile. In Teradata, this can be done by joining DBC.QryLogObjects to DBC.QryLog and selecting the object database names referenced by the target user population.

The following query was used to populate the scope_databases table for review:

```
INSERT INTO benchmark.scope_databases
    (database_name, in_scope_flag)
SELECT DISTINCT
    qlo.ObjectDatabaseName,
    'Y'
FROM dbc.qrylogobjects qlo
JOIN dbc.qrylog ql
  ON qlo.queryid = ql.queryid
WHERE ql.UserName LIKE 'OSB%'
  AND qlo.ObjectType = 'DB'
  AND CAST(ql.CollectTimeStamp AS DATE)
     BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'
  AND CAST(qlo.CollectTimeStamp AS DATE)
      BETWEEN DATE '2026-02-01' AND DATE '2026-02-28';
```

This query provides an initial list of databases referenced by the selected user population. The resulting rows should then be reviewed to identify databases that are outside the intended scope of the information system, such as:

- personal databases
- work databases
- temporary or staging areas
- unrelated application databases

Those databases can then be updated in the scope_databases table by setting in_scope_flag to 'N'.

The scope_databases table becomes a control table used throughout the remainder of the workload selection and preparation process.

Appendix C — Populating and Profiling selected_queries

The selected_queries table is populated from the activity hours selected from the workload profile. In this example, queries are selected from four representative hours in February for the OSB user population.

The selected_queries table should be reviewed using a SQL GUI tool such as DBeaver. Sort the rows by session_id and query_seq to view each session in execution order. As you review the SQL, update the include_flag to include or exclude queries based on the screening criteria, allowing you to iteratively refine the workload.

If historical query logs are stored in Teradata PDCR tables, a logdate predicate should be added for each range to take advantage of partition elimination.

Populating selected_queries

The following query populates the initial contents of selected_queries:

```
INSERT INTO benchmark.selected_queries
(   session_id,
    logon_ts,
    query_id,
    query_seq,
    sql_text
)
SELECT ql.SessionID
    , ql.LogonDateTime
    , ql.QueryID
    , ql.RequestNum
    , qls.SqlTextInfo
FROM DBC.QRYLOG ql
JOIN DBC.QryLogSQL qls
  ON ql.QueryID = qls.QueryID
WHERE ql.UserName LIKE 'OSB%'
  AND (
         ql.LogonDateTime BETWEEN TIMESTAMP '2025-02-04 10:00:00'
                              AND TIMESTAMP '2025-02-04 11:00:00'
      OR ql.LogonDateTime BETWEEN TIMESTAMP '2025-02-09 14:00:00'
                              AND TIMESTAMP '2025-02-09 15:00:00'
      OR ql.LogonDateTime BETWEEN TIMESTAMP '2025-02-14 10:00:00'
                              AND TIMESTAMP '2025-02-14 11:00:00'
      OR ql.LogonDateTime BETWEEN TIMESTAMP '2025-02-20 11:00:00'
                              AND TIMESTAMP '2025-02-20 12:00:00'
      );
```

After the initial insert, the selected_queries table can be reviewed and refined iteratively. Some exclusions can be made in bulk before manual review.

One example is the DATABASE statement. It is often issued at the start of a session to establish the default database, creating a dependency for all subsequent queries in that session. Because this setting cannot be executed independently of those queries, it should not be treated as a separate work unit. Instead, the default database can be set once per session using the TrueBench BEFORE_WORKER statement, allowing dependent queries to execute correctly without retaining the original statement.

The following example updates those rows to exclude them from further consideration:

```
UPDATE benchmark.selected_queries
SET include_flag = 'N',
    review_note  = 'Excluded: handled by BEFORE_WORKER'
WHERE UPPER(TRIM(sql_text)) LIKE 'DATABASE %';
```

Additional bulk update statements can be used to exclude other known categories of noise or out-of-scope activity before the detailed session review begins.

After bulk filtering and manual review, the selected rows should be re-profiled and compared to the original production profile to confirm that the remaining workload still reflects production behavior.

Profiling selected_queries

The following queries are revised versions of the profiling SQL from TBM-02. In each case, the production workload is constrained using the selected_queries table, and include_flag = 'Y', allowing the selected workload to be compared iteratively to the original production profile as sessions and queries are added or removed.

Table Utilization for Selected Workload

This version restricts table access profiling to the queries currently included in selected_queries.

```
WITH RawUsage AS
(
    SELECT
        qlo.QueryID,
        qlo.ObjectDatabaseName,
        qlo.ObjectTableName,
        qlo.TargetIndicator
    FROM dbc.qrylogobjects qlo
```

```
    JOIN benchmark.selected_queries sq
      ON qlo.QueryID = sq.query_id
    WHERE sq.include_flag = 'Y'
      AND qlo.ObjectType = 'Tab'
    GROUP BY 1,2,3,4
)
SELECT
    ObjectDatabaseName,
    ObjectTableName,
    TargetIndicator AS WriteMode,
    COUNT(DISTINCT QueryID) AS QueryCount
FROM RawUsage
GROUP BY ObjectDatabaseName, ObjectTableName, WriteMode
ORDER BY QueryCount DESC;
```

Query Complexity for Selected Workload

This version profiles only the included queries in selected_queries.

```
WITH RawComplexity AS
(
  SELECT
     ql.QueryID,
     ql.NumSteps,
     CAST(LOG(NULLIFZERO(ql.NumResultRows)) AS INTEGER) AS
LogRowsOut,
     COUNT(DISTINCT qlo.ObjectDatabaseName ||
qlo.ObjectTableName) AS TableCount
  FROM benchmark.selected_queries sq
  JOIN dbc.qrylog ql
    ON sq.query_id = ql.QueryID
  JOIN dbc.qrylogobjects qlo
    ON ql.QueryID = qlo.QueryID
  WHERE sq.include_flag = 'Y'
    AND qlo.ObjectType = 'Tab'
  GROUP BY 1, 2, 3
)
SELECT
    NumSteps,
    TableCount,
    LogRowsOut,
    COUNT(*) AS QueryCount
FROM RawComplexity
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

Appendix D — Identifying Executable Work Units

The purpose of this step is to transform the refined contents of selected_queries into executable work units in selected_work.

The process begins with a simple assumption: queries are independent unless there is evidence that they must be preserved together as part of a session sequence.

The examples below show how to:

- assign standalone work units
- group sessions containing volatile-table activity into multi-step work units
- identify sessions using work tables for review

These examples are intended as a starting point. Manual review may still be required for sessions with more complex dependency patterns.

D.1 Initialize selected_work as Standalone Queries

The first pass assumes that each included query can be executed independently.

```
INSERT INTO benchmark.selected_work
(
    work_id,
    session_id,
    logon_ts,
    query_id,
    query_seq,
    work_seq,
    sql_text,
    work_type,
    include_flag,
    review_status
)
SELECT
    ROW_NUMBER() OVER (ORDER BY session_id, logon_ts, query_seq)
AS work_id,
    session_id,
    logon_ts,
    query_id,
    query_seq,
    1 AS work_seq,
    sql_text,
```

```
    'STANDALONE' AS work_type,
    include_flag,
    'AUTO_INCLUDED' AS review_status
FROM benchmark.selected_queries
WHERE include_flag = 'Y';
```

This provides an initial one-query-per-work-unit structure.

D.2 Identify Sessions Containing Volatile Table Activity

If a session creates a volatile table, the full session should be treated as a single work unit.

The following query identifies those sessions:

```
SELECT DISTINCT
    session_id,
    logon_ts
FROM benchmark.selected_queries
WHERE include_flag = 'Y'
  AND UPPER(sql_text) LIKE '%CREATE VOLATILE TABLE%';
```

D.3 Rebuild Volatile-Table Sessions as Multi-Step Work Units

The standalone assignments for those sessions can be removed and replaced with a single ordered multi-step sequence.

```
DELETE FROM benchmark.selected_work sw
WHERE EXISTS
(
    SELECT 1
    FROM benchmark.selected_queries sq
    WHERE sq.session_id   = sw.session_id
      AND sq.logon_ts     = sw.logon_ts
      AND sq.include_flag = 'Y'
      AND UPPER(sq.sql_text) LIKE '%CREATE VOLATILE TABLE%'
);

INSERT INTO benchmark.selected_work
(
    work_id,
    session_id,
    logon_ts,
    query_id,
```

```
    query_seq,
    work_seq,
    sql_text,
    work_type,
    include_flag,
    review_status
)
SELECT
    100000 + DENSE_RANK() OVER (ORDER BY session_id, logon_ts) AS
work_id,
    session_id,
    logon_ts,
    query_id,
    query_seq,
    ROW_NUMBER() OVER
        (PARTITION BY session_id, logon_ts ORDER BY query_seq) AS
work_seq,
    sql_text,
    'MULTISTEP' AS work_type,
    include_flag,
    'AUTO_INCLUDED' AS review_status
FROM benchmark.selected_queries
WHERE include_flag = 'Y'
  AND (session_id, logon_ts) IN
      (
          SELECT DISTINCT session_id, logon_ts
          FROM benchmark.selected_queries
          WHERE include_flag = 'Y'
            AND UPPER(sql_text) LIKE '%CREATE VOLATILE TABLE%'
      );
```

If your DBMS does not support row-value predicates such as (session_id, logon_ts) IN (...), rewrite that condition as an EXISTS subquery.

The offset of 100000 is only an example to keep these generated multi-step work identifiers separate from the initial standalone assignments.

D.4 Flag Queries Referencing Out-of-Scope or Work Databases

The scope_databases table can be used to automatically flag queries that reference databases outside the information system, as well as databases used for work and staging.

- Databases marked **N (out of scope)** are excluded

- Databases marked **W (work/staging)** are excluded initially and flagged for review

```
UPDATE benchmark.selected_work sw
FROM
(
    SELECT
        sw.query_id,
        MAX(CASE WHEN sd.in_scope_flag = 'N' THEN 1 ELSE 0 END)
AS has_n_scope,
        MAX(CASE WHEN sd.in_scope_flag = 'W' THEN 1 ELSE 0 END)
AS has_w_scope
    FROM benchmark.selected_work sw
    JOIN dbc.qrylogobjects qlo
      ON sw.query_id = qlo.queryid
    JOIN benchmark.scope_databases sd
      ON qlo.ObjectDatabaseName = sd.database_name
    WHERE sw.include_flag = 'Y'
      AND qlo.ObjectType = 'DB'
      AND sd.in_scope_flag IN ('N', 'W')
    GROUP BY sw.query_id
) x
SET include_flag =
        CASE
            WHEN x.has_n_scope = 1 THEN 'N'
            WHEN x.has_w_scope = 1 THEN 'N'
            ELSE sw.include_flag
        END,
    review_status =
        CASE
            WHEN x.has_n_scope = 1 THEN 'AUTO_EXCLUDED'
            WHEN x.has_w_scope = 1 THEN 'REVIEW_REQUIRED'
            ELSE review_status
        END,
    review_note =
        CASE
            WHEN x.has_n_scope = 1 THEN 'Excluded: references
out-of-scope database'
            WHEN x.has_w_scope = 1 THEN 'Review: references work
database'
            ELSE review_note
        END
WHERE sw.query_id = x.query_id
  AND sw.include_flag = 'Y';
```

This approach uses SQL text matching to identify database references. It is intended to support practical filtering and review.

D.5 Review and Finalize Query Selection and Work Units

Queries flagged with review_status = 'REVIEW_REQUIRED' should be reviewed using a SQL GUI tool such as DBeaver.

Sort rows by:

- session_id
- query_seq

This allows each session to be reviewed in execution order, so that dependencies and sequencing are clearly understood.

During review, consider whether:

- the referenced tables are intermediate structures rather than production tables
- the session should be preserved as a multi-step sequence
- worktables can be converted to volatile tables for repeatable execution
- queries can be treated as independent or must remain grouped

Based on this review:

- update include_flag to include or exclude queries
- assign or adjust work_id and work_seq for multi-step sequences
- update work_type as STANDALONE or MULTISTEP
- document decisions using review_status and review_note

This step allows sessions that rely on worktables or intermediate processing to be retained where appropriate, while ensuring the resulting workload remains repeatable and suitable for concurrent execution.

D.6 Summarize the Current Workload

After the initial work-unit assignments and any manual review, summarize the contents of selected_work to confirm the current workload size and composition.

The following query reports the number of sessions, work units, and queries by include_flag and work_type, allowing standalone and multi-step work to be reviewed separately:

```
SELECT
    include_flag,
    work_type,
    COUNT(DISTINCT session_id || '|' || CAST(logon_ts AS
VARCHAR(30))) AS SessionCount,
    COUNT(DISTINCT work_id) AS WorkUnitCount,
    COUNT(*) AS QueryCount
FROM benchmark.selected_work
GROUP BY include_flag, work_type
ORDER BY include_flag, work_type;
```

This summary helps confirm whether the workload is at a practical size before additional reduction is applied.

D.7 Reduce Session Volume Using Modulo-Based Sampling

Even after screening and work-unit definition, the number of sessions may still be larger than is practical to prepare.

Sessions can be reduced in a controlled way by ordering them by resource usage (e.g., CPU), assigning a session number, and excluding approximately every *n*th session. This reduction is applied at the **session level**, preserving multi-step sequences and BI-generated sessions.

The following example updates selected_work to exclude sessions based on modulo sampling:

```
UPDATE benchmark.selected_work sw
FROM
(
    SELECT
        ordered_sessions.session_id,
        ordered_sessions.logon_ts
    FROM
    (
        SELECT
            sw.session_id,
            sw.logon_ts,
            SUM(ql.AMPCPUTime / 100.0) AS session_cpu,
            ROW_NUMBER() OVER
            (
                ORDER BY SUM(ql.AMPCPUTime / 100.0) DESC,
                         sw.session_id,
```

```
                        sw.logon_ts
                ) AS cpu_order_seq
            FROM benchmark.selected_work sw
            JOIN dbc.qrylog ql
              ON sw.query_id = ql.queryid
            WHERE sw.include_flag = 'Y'
            GROUP BY sw.session_id, sw.logon_ts
        ) ordered_sessions
        WHERE MOD(ordered_sessions.cpu_order_seq, 5) <> 1
    ) d
    SET include_flag  = 'N',
        review_status = 'AUTO_EXCLUDED',
        review_note   = 'Excluded by modulo-based session reduction'
    WHERE sw.session_id = d.session_id
      AND sw.logon_ts   = d.logon_ts
      AND sw.include_flag = 'Y';
```

In this example, only sessions where MOD(session_no, 5) = 1 are retained, keeping approximately one-fifth of the sessions. The modulus value can be adjusted based on the required reduction.

D.8 Synchronize selected_queries and Re-Profile

After manual review or session reduction in selected_work, update selected_queries so that the profiling queries in Appendix C reflect the current set of included work units.

The following example updates selected_queries based on the current include_flag values in selected_work:

```
UPDATE benchmark.selected_queries sq
FROM
(
    SELECT
        query_id,
        MAX(include_flag) AS include_flag
    FROM benchmark.selected_work
    GROUP BY query_id
) sw
SET sq.include_flag = sw.include_flag
WHERE sq.query_id = sw.query_id;
```

After synchronizing the include_flag, rerun the Appendix C profiling queries using selected_queries to confirm that the workload continues to reflect the characteristics of the production profile after refinement.

This process of reviewing selected_work, synchronizing selected_queries, and re-profiling continues until the workload is both practical to prepare and acceptable as a representation of production activity.

Appendix D Notes

These examples use simple text matching to identify likely dependency patterns. They are intended to support a practical review process, not to guarantee complete automatic classification. In particular, sessions that create parameter tables in a default database or use more complex intermediate structures may require manual inspection.